



ORACLE

MySQL Implementation Essentials Bootcamp

Database Design

Marco Carlessi (marco.carlessi@oracle.com)
Andreea Ciobotaru (andreea.Ciobotaru@oracle.com)
MySQL Sales Consulting

2023



Program Agenda

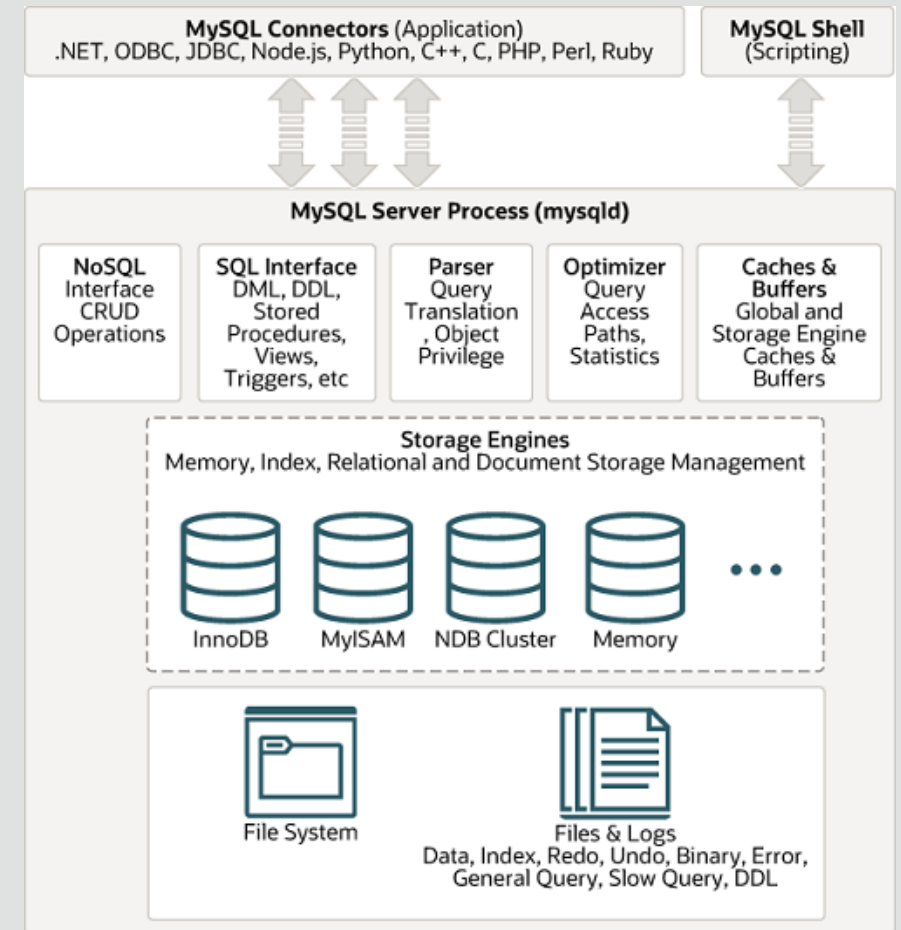
- Introduction
- Oracle MySQL Enterprise
- Oracle Cloud Infrastructure overview
- Installation and Architecture
- **Database Design**
- Security
- Backup
- Replication
- High Availability
- MySQL Enterprise Monitor
- HeatWave

MySQL Database design

- In MySQL "Schema" and "Database" are synonym
- On the file system
 - a database is an OS directory
 - tables are files
- There are some internally created schemas
 - data dictionary: mysql, information_schema
 - performances: performance_schema, sys
 - InnoDB Cluster metadata: mysql_innodb_cluster_metadata

Storage Engines

- MySQL server architecture isolates the application programmer and DBA from all of the low-level implementation details at the storage level
- Some of the key feature differentiations include:
 - Concurrency
 - Transaction Support
 - Referential Integrity
 - Physical Storage
 - Index Support
 - Memory Caches
 - Performance Aids
 - Miscellaneous Target Features



MySQL InnoDB

- InnoDB is a general-purpose storage engine that balances high reliability and high performance
- InnoDB is **ACID** compliant and **fully transactional** with ROLLBACK/COMMIT/Foreign Keys
- A typical MySQL database is normally in the lower three-digit GB
 - Theoretical limit of the **maximum tablespace size is 64TB**
 - It can serve more than **1000 columns** per table
 - Contain a maximum of 64 **secondary indexes**
 - A row can have a size of up to 64Kb (less BLOBs)
- InnoDB supports encryption and compression
- Built in HA Clustering available (Group Replication)

MySQL InnoDB: transactions

- By default autocommit=ON
- The statements "Start Transaction" and "Begin" are synonym
- A "start transaction" statement temporarily disables autocommit (for DML)

- Example:

```
begin; [start transaction]
create table t1 (c1 int);
create table t2 (c1 int);
rollback;
create table t3 (c1 int);
show tables;
```

```
start transaction; [begin]
INSERT INTO t1 ( ` name ` ) VALUES ('Jonas');
INSERT INTO t1 ( ` name ` ) VALUES ('Lisa');
rollback;
INSERT INTO t1 ( ` name ` ) VALUES ('Beth');
select * from c1;
```

MySQL Tables

- Syntax

CREATE [TEMPORARY] TABLE [IF NOT EXISTS] tbl_name (create_definition,...) [table_options] [partition_options]

- e.g.

```
CREATE TABLE `ppt_test`.`ppt_table_test`  
(  
  `id` INT NOT NULL AUTO_INCREMENT,  
  `lastname` VARCHAR(45) NULL,  
  `forename` VARCHAR(45) NULL,  
  `birthday` DATE NULL,  
  PRIMARY KEY (`id`));
```

- **ANALYZE TABLE** performs a key distribution analysis and stores the distribution for the named table or tables
 - removes the table from the table definition cache, which requires a flush lock. If there are long running statements or transactions still using the table, subsequent statements and transactions must wait for those operations to finish before the flush lock is released
 - The persistent optimizer statistics feature (enabled by default) improves plan stability by storing statistics to disk and making them persistent across server restarts
 - statistics are calculated automatically when a table undergoes changes to more than 10% of its rows (enabled by default, configurable)
- **OPTIMIZE TABLE** reorganizes the physical storage of table data and associated index data, to reduce storage space and improve I/O efficiency when accessing the table

MySQL Data Types

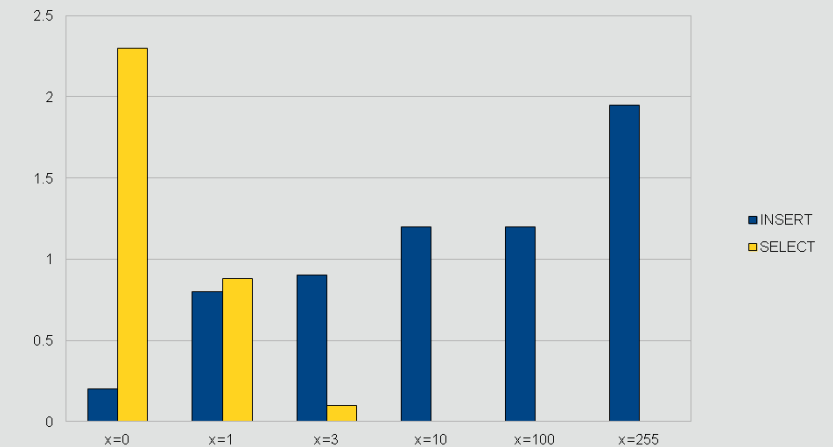
- Each data type has different ranges, defaults and storage requirements
- MySQL provides also dedicated functions
- **Numeric exact value**
 - INTEGER, INT, SMALLINT, TINYINT, MEDIUMINT, BIGINT
 - DECIMAL, NUMERIC
- **Numeric approximate value**
 - FLOAT, DOUBLE
- **Bit-Value Type**
 - BIT
- **Date and Time Types**
 - DATE, DATETIME, TIMESTAMP, YEAR
- **String Types**
 - CHAR, VARCHAR
 - BINARY, VARBINARY
 - ENUM
 - SET
- **BLOB**
 - TINYBLOB, BLOB, MEDIUMBLOB, and LONGBLOB
- **TEXT**
 - TINYTEXT, TEXT, MEDIUMTEXT, and LONGTEXT
- **Spatial**
 - GEOMETRY, POINT, LINESTRING, POLYGON
 - MULTIPOINT, MULTILINESTRING, MULTIPOLYGON, GEOMETRYCOLLECTION
- **JSON Data Type**

Type	Storage Required	Signed Range	Value	CHAR (4)	Storage Required	VARCHAR (4)	Storage Required
			' '	' '	4 bytes	' '	1 byte
FLOAT	4 bytes	-3.402823466E+38 to -1.1751175494351E-38 to 3.402823466E+38 to 1.1751175494351E-38	'ab'	'ab'	4 bytes	'ab'	3 bytes
			'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
DOUBLE	8 bytes	-1.7976931348623157E+308 to -2.2250738585072014E-308 to 1.7976931348623157E+308 to 2.2250738585072014E-308	'abcdefgh'	'abcd'	4 bytes	'abcd'	5 bytes

MySQL Indexes

- Optimize the common lookup and DML operations
 - BTREE
 - HASH
 - RTREE (spatial only)
- Automatic storage of data in a sorted order
 - Non-Unique - May refer to many records
 - Unique or Primary - Must refer to only one record
- Special cases
 - Full-Text (CHAR, VARCHAR, or TEXT)
 - Multi-value (JSON only)
- Prefix indexes
 - For string columns, indexes can be created that use only the leading part of column values
`CREATE INDEX name ON customer(COL1(10)) ;`
- Compound indexes

```
index key123 (col1,col2,col3)
index key12 (col1,col2)      <- Not needed!
index key1 (col1)           <- Not needed!
```



Indexes: tips

- Create a Primary Key on each table
- PK is replicated in each secondary index
 - Keep Primary Key simple
- Too many indexes can slow down inserts/deletes
 - Use only the indexes you must have
 - Check often if they are used or not (use sys report or MEM)
- Reduce indexes using prefix and compound keys
- For large strings, consider using a separate column using the MySQL MD5 to create a hash key and index on it instead

Examples

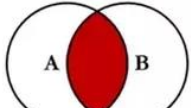
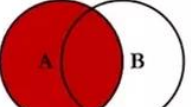
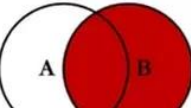
```
Create table address (  
    id INT unsigned not null,  
    lname char(30),  
    fname char(30),  
    Index name (lname, fname)  
);
```

- A. `select * from address where lname='peter';`
- B. `select * from address where lname='peter' and fname='hase';`
- C. `select * from address where lname='peter' or fname='hase';`
- D. `select * from address where fname='hase' and id>0;`

Joining Tables

<https://dev.mysql.com/doc/refman/8.0/en/join.html>

- **Join (Inner Join)**
 - return rows from either table to appear in the result if and only if both tables meet the conditions specified in the ON clause
- **Left Join (Outer Join)**
 - returns all records from the left table, and the matched records from the right table
- **Right Join (Outer Join)**
 - returns all records from the right table, and the matched records from the left table

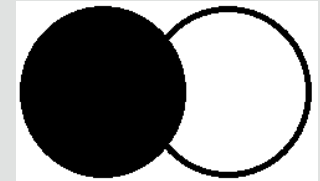
Table A = Employees (EmployeeID, LastName, ...) ^ Table B = Orders (OrderID, EmployeeID, ...)	
 <pre>SELECT <select_list> FROM TableA A INNER JOIN TableB B ON A.Key = B.Key</pre>	Employees who have registered orders <pre>SELECT Employees.LastName, Count(Orders.OrderID) AS NumberOfOrders FROM Employees INNER JOIN Orders ON Employees.EmployeeID=Orders.EmployeeID GROUP BY Employees.LastName</pre>
 <pre>SELECT <select_list> FROM TableA A LEFT JOIN TableB B ON A.Key = B.Key</pre>	All employees and orders they have registered <i>(There are employees who have not registered any orders)</i> <pre>SELECT Employees.LastName, Count(Orders.OrderID) AS NumberOfOrders FROM Employees LEFT JOIN Orders ON Employees.EmployeeID=Orders.EmployeeID GROUP BY Employees.LastName</pre>
 <pre>SELECT <select_list> FROM TableA A RIGHT JOIN TableB B ON A.Key = B.Key</pre>	All orders and employees who registered them <i>(There are orders that have not been registered by any employee)</i> <pre>SELECT Employees.LastName, Orders.OrderID FROM Employees RIGHT JOIN Orders ON Employees.EmployeeID=Orders.EmployeeID</pre>

Examples: Joins (Outer)

- All employees and their registered orders:

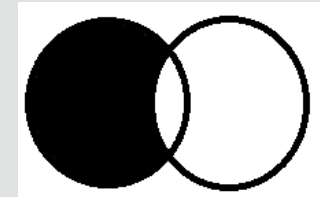
```
SELECT * FROM emp LEFT JOIN orders ON emp.id=orders.emp_id;
```

emp / orders



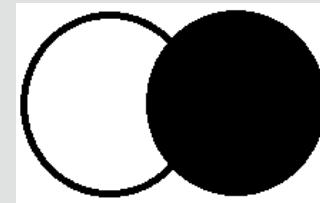
- Employees who have not registered any orders:

```
SELECT * FROM emp LEFT JOIN orders ON emp.id=orders.emp_id  
WHERE orders.emp_id IS NULL;
```



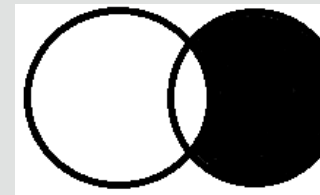
- All orders and employees who have registered them:

```
SELECT * FROM emp RIGHT JOIN orders ON  
emp.id=orders.emp_id;
```



- All orders that do not have a employee:

```
SELECT * FROM emp RIGHT JOIN orders ON emp.id=orders.emp_id  
WHERE emp.id IS NULL;
```



MySQL Partitioning

<https://dev.mysql.com/doc/refman/8.0/en/partitioning.html>

- The partition plugin enables to distribute portions of individual tables across a file system according to rules
 - Ranges, lists, hash, key
- MySQL allows distribution of partitions across an instance/physical storage
- Explain return info on partition usage in a query
`explain select * from world_innodb.City_part;`
- Support for partitioning & HA are not part of the MySQL Standard version (only Enterprise Edition)

```
mysql> SELECT
->     PLUGIN_NAME as Name,
->     PLUGIN_VERSION as Version,
->     PLUGIN_STATUS as Status
-> FROM INFORMATION_SCHEMA.PLUGINS
-> WHERE PLUGIN_TYPE='STORAGE ENGINE';
```

Name	Version	Status
binlog	1.0	ACTIVE
CSV	1.0	ACTIVE
MEMORY	1.0	ACTIVE
MRG_MYISAM	1.0	ACTIVE
MyISAM	1.0	ACTIVE
PERFORMANCE_SCHEMA	0.1	ACTIVE
BLACKHOLE	1.0	ACTIVE
ARCHIVE	3.0	ACTIVE
InnoDB	5.7	ACTIVE
partition	1.0	ACTIVE

10 rows in set (0.00 sec)

EXPLAIN

- EXPLAIN show it would process the statement, including information about how tables are joined and in which order
 - When EXPLAIN is used with FOR CONNECTION connection_id rather than an explainable statement, it displays the execution plan for the statement executing in the named connection
 - works with SELECT, DELETE, INSERT, REPLACE, and UPDATE statements
- **References**
 - <https://dev.mysql.com/doc/refman/8.0/en/explain.html>
 - <https://dev.mysql.com/doc/refman/8.0/en/explain-output.html>
- EXPLAIN ANALYZE produces EXPLAIN output along with timing and additional, iterator-based, information about how the optimizer's expectations matched the actual execution



Hands-On Labs



3a. Database Design

JSON Data

- JSON (JavaScript Object Notation) defined by RFC 7159
 - Open standard file format that uses human-readable text
- It use Objects and Arrays
- Examples

```
"firstName": "John"
```

```
{"firstName": "John", "lastName": "Doe"}
```

```
"employess": [  
  {"firstName": "John", "lastName": "Doe"},  
  {"firstName": "Anna", "lastName": "Smith"},  
  {"firstName": "Peter", "lastName": "Jones"}  
]
```

JSON: Why is it popular?

- Developer Friendly
 - Simple data format that allows programmers to store and communicate sets of values
 - JSON's lack of a predefined schema makes it easy for developers to store and update documents
- Its ease of use, speed of processing and easy integration made it extremely popular for Web-based applications
- As JSON adoption has grown, JSON-centric document databases have become more popular
- MySQL 5.7 integrated JSON features, resulting in a best-of-both worlds benefit to developers and database administrators

Core New JSON features in MySQL 5.7

- **Native JSON data type**

- Native internal **binary format** for efficient processing & storage
- Up to 10x faster than storing as text
- Provides Document Validation

```
INSERT INTO employees VALUES ('some random text');
```

```
ERROR 3130 (22032): Invalid JSON text: "Expect a value here." at position 0 in value (or column) 'some random text'.
```

- **Built-in JSON functions**

- Allowing you to efficiently store, search, update, and manipulate Documents

- **Indexing of Documents using Generated Columns (Indirect)**

- Automatically uses the best “functional” index available for even faster results
- Key/Array references enable quick read-access to look up document elements directly

- **JSON Comparator & New inline syntax for easy SQL integration**

- Allows for easy integration of Document data within your SQL queries

JSON Functions

- 5.7/8.0 supports functions to CREATE, SEARCH, MODIFY and RETURN JSON values

JSON_ARRAY_APPEND()

JSON_ARRAY_INSERT()

JSON_ARRAY()

JSON_CONTAINS_PATH()

JSON_CONTAINS()

JSON_DEPTH()

JSON_EXTRACT()

JSON_INSERT()

JSON_KEYS()

JSON_LENGTH()

JSON_MERGE()

JSON_OBJECT()

JSON_QUOTE()

JSON_REMOVE()

JSON_REPLACE()

JSON_SEARCH()

JSON_SET()

JSON_TYPE()

JSON_UNQUOTE()

JSON_VALID()

JSON_PRETTY()

JSON_STORAGE_SIZE()

JSON_STORAGE_FREE()

JSON_ARRAYAGG()

JSON_OBJECTAGG()

JSON_TABLE()

JSON reference

```
1 • |select JSON_EXTRACT(data,'$.name') from employees;
2
```

```
1 • CREATE TABLE employees (data JSON);
2 • INSERT INTO employees VALUES ('{"id": 1, "name": "Jane"}');
3 • INSERT INTO employees VALUES ('{"id": 2, "name": "Joe"}');
4 • INSERT INTO employees VALUES ('{"id": 3, "name": "Peter"}');
5 • INSERT INTO employees VALUES ('{"id": 4, "name": "Anna"}');
6 • INSERT INTO employees VALUES ('{"id": 5, "name": "Henry"}');
7
8 • SELECT * FROM employees;
9
```

data
{ "id": 1, "name": "Jane" }
{ "id": 2, "name": "Joe" }
{ "id": 3, "name": "Peter" }

PATH Expression – internal pointer
address of a value inside a JSON document
Shortcut: **data->\$name**

JSON_EXTRACT(data,'\$.name')
"Jane"
"Joe"
"Peter"
"Anna"
"Henry"

Introducing Generated Columns

- Columns computed from an expression included in the column definition
 - VIRTUAL: computed when read, not stored, indexable
 - STORED: computed when inserted/updated, stored in SE, indexable

<u>id</u>	my_integer	my_integer_plus_one
1	10	11
2	20	21
3	30	31
4	40	41

Column automatically maintained based on your specification.

```
CREATE TABLE t1 (  
  id INT NOT NULL PRIMARY KEY auto_increment,  
  my_integer INT,  
  my_integer_plus_one INT AS (my_integer+1)  
);
```

```
UPDATE t1 SET my_integer_plus_one = 10 WHERE id = 1;  
ERROR 3105 (HY000): The value specified for generated column  
'my_integer_plus_one' in table 't1' is not allowed.
```

Read-only of course

Generated Columns Support Indexes!

Meta data change only (FAST). Does not need to touch table

```
ALTER TABLE features ADD feature_type VARCHAR(30) AS (feature->"$.type");
```

```
ALTER TABLE features ADD INDEX (feature_type);
```

Creates index only. Does not modify table rows

```
SELECT feature FROM features WHERE feature_type = "feature";
```

...

```
SELECT feature FROM features WHERE feature->"$.type" = "feature";
```

...

Optimizer automatically recognizes Index expressions

DBMS or NoSQL ?

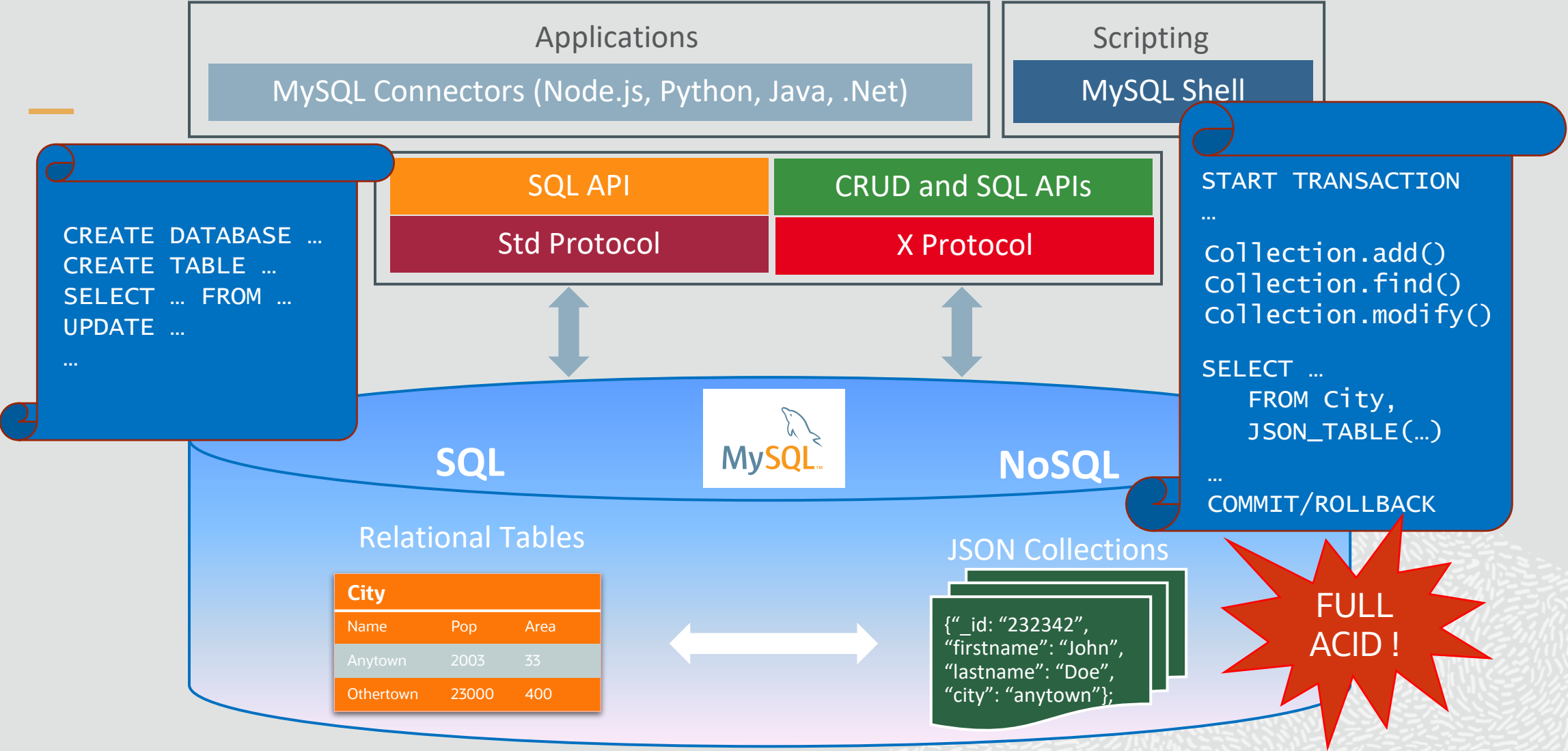


Why not both ?

<https://www.youtube.com/watch?v=1Dk517M-7o>



MySQL 8.0 Document Store: SQL+NoSQL database



Tables or Collections?

- A collection is a table with 2+ columns:
 - Primary key: `\_id`
 - JSON document: `doc`
- The document's `\_id` field could be supplied or automatically generated as UUID
 - This field could be also used to populate the primary key
- Can add extra columns and indexes to a collection
- SQL, NoSQL, tables, collections, all can be used simultaneously
- Operations compatible with replication

X DevAPI

- Use SQL, CRUD APIs – Document (NoSQL) and Relational (SQL), or “All of the Above”
 - All of this is in addition to the Classic APIs
- Implemented in connectors for
 - C++, Java, .Net, Node.js, Python, PHP
 - working with Communities
- Non-blocking, asynchronous calls follow common language patterns
- Supports CRUD operations

Operation	Document	Relational
Create	Collection.add()	Table.insert()
Read	Collection.find()	Table.select()
Update	Collection.modify()	Table.update()
Delete	Collection.remove()	Table.delete()

MySQL Document Store cheat sheet

```
js> session.createSchema('name')
js> \use name
js> db.getCollections()
js> db.createCollection('myCollection')
js> db.getCollections()
js> db.myCollection.add({"param1":"value1", "param2":"value2"})
```

Create

```
js> db.myCollection.find()
js> db.myCollection.find().limit(1)
js> db.myCollection.find("_id = '00005af0184300000000000000000002'")
```

Read

```
js> db.myCollection.modify("_id = '1234'").set("param","value")
```

Update

```
js> db.myCollection.remove("_id = '1234'")
```

Delete

```
js> session.startTransaction()
js> ...
js> session.rollback()
```

Transactions



Hands-On Labs



3b. MySQL JSON datatype

3c. MySQL Document Store



ORACLE